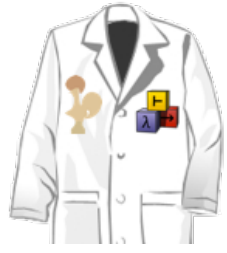
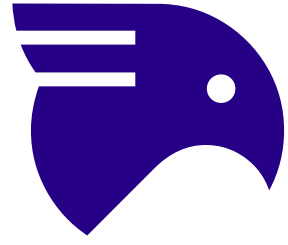
An EVM Specification for the Rocq Prover using Mathematical Components: Work in Progress

Karl Palmskog

KTH Royal Institute of Technology, Stockholm, Sweden





The Rocq Prover

Rocq Prover (`https://rocq-prover.org`):

- programming environment and prover with dependent types
- known as Coq until 8.20, new name from 9.0
- large ecosystem spanning mathematics and computer science
- popular for programming language semantics and metatheory
- supports extraction to OCaml, compilation to C & RISC-V
- developed for the long term by Inria, CNRS, volunteers
- biannual major version releases with backwards compatibility

The Mathematical Components (MathComp) Library

MathComp (`https://math-comp.github.io`):

- library of basic algebraic structures and beyond
- more coherently designed and organized than Rocq's Stdlib
- based on canonical structures for structure inference
- supported by Hierarchy Builder tool
- optimized for easy proofs by rewriting & boolean reflection
- ecosystem with applications across math/CS

```
Theorem Feit_Thompson
  (gT : finGroupType) (G : {group gT}) :
  odd #|G| -> solvable G.
Proof. exact: (minSimpleOdd_ind no_minSimple_odd_group).
Qed.
```

Our Goal: Validated Rocq EVM using MathComp

- want EVM definition based on MathComp best practices
- focus on ease-of-reasoning rather than execution
- maintained as Rocq and MathComp and the EVM evolves
- validation by use in verified compilation, metatheory, ...
- aim for biannual releases to match major Rocq versions

mcmword: Machine Words in Rocq using MathComp

- mathcomp-word library/package introduces basic MathComp structures and utility functions for abstract words
- Jasmin compiler formalization extends for machine words
- mcmword library: adaptation/packaging of Jasmin's approach
- aims to also host MathComp-style abstract cryptographic function formalizations such as SHA256, Keccak
- licensed under MIT like mathcomp-word/Jasmin

```
Definition sig0 (w: u32) : u32 :=
 wxor (wror w 7) (wxor (wror w 18) (wshr w 3)).

Definition sha256msg1 (m1 m2: u128) : u128 :=
  let s := split_vec VE32 m1 in
  make_vec U128 (map2 (fun x y, x + sig0 y) s
   (rcons (behead s) (zero_extend U32 m2))).
```

mcevm: EVM in Rocq using MathComp (WIP)

- built on MathComp and mcmword, works with Rocq 9.0
- comparison point: export of Lem EVM by Yoichi Hirai to Rocq
- work in progress (incomplete, needs validation)
- mcmword co-developed in the same repository on GitHub

Public preview: `https://github.com/palmskog/mcevm`

Refinement and Executability

- MathComp code is typically not efficient or executable
- Peregrine allows obtaining verified OCaml/C from Rocq code
- aim to use Trocq plugin for automated refinement
- `https://peregrine-project.github.io`
- `https://github.com/rocq-community/trocq`

```
From Peregrine.Plugin Require Import Loader.
Peregrine Extract "prog.ast" prog.
```

mcevm Code Fragments

```
#[only(eqbOK)] derive
Variant sarith_inst :=
| SDIV   (* signed division *)
| SMOD   (* signed modulo *)
| SGT    (* signed greater-than *)
| SLT    (* signed less-than *)
| SIGNEXTEND.
HB.instance Definition _ :=
 hasDecEq.Build sarith_inst sarith_inst_eqb_OK.
```

mcevm Code Fragments

```
Definition stack_inst_code (si:stack_inst) : seq byte :=
match si with
| POP => [::wrepr U8 80]
| PUSH_N lst =>
    if (size lst < 1)%nat
    then [::wrepr U8 96; wrepr U8 0]
    else if (size lst > 32)%nat
    then [::wrepr U8 96; wrepr U8 0]
    else [::(wrepr U8 (Z.of_nat (size lst)) +
      wrepr U8 95)%R] ++ lst
| CALLDATALOAD => [::wrepr U8 53]
end.
```